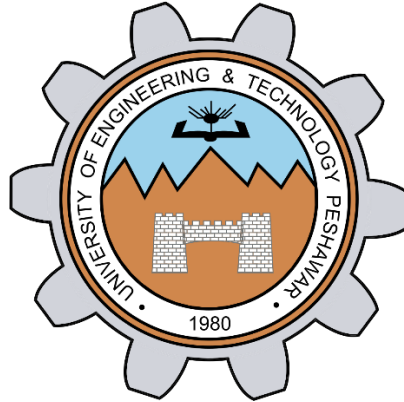


**University of Engineering and Technology Peshawar
(Jalozai Campus)**

Department of CS&IT



File Search Tool in Linux

By

Muhammad Ahmad Khan (23JZBC0238)

Aleena Khan (23JZBCS0229)

Faizan Ullah Wazir (23JZBCS042)

Rabia Qaiser Khan (23JZBCS0245)

Table of Content

1	PROJECT TITLE.....	1
2	INTRODUCTION.....	1
3	OBJECTIVES.....	1
4	FEATURES.....	2
5	REQUIREMENTS.....	2
6	INSTALLATION.....	3
7	USAGE.....	5
7.1	RUN THE APPLICATION.....	5
7.2	ENTERING SEARCH PARAMETERS.....	5
7.3	SEARCH FOR THE FILE.....	6
8	TROUBLESHOOTING.....	6
8.1	EXECUTABLE NOT FOUND ERROR.....	6
8.2	PERMISSION DENY.....	6
8.3	NO OUTPUT ERROR.....	6
8.4	GUI ERROR.....	7
8.5	SEGMENTATION FAULT.....	7
9	CONCLUSION.....	7
10	CODE AND SNAPSHOT.....	8
10.1	BACKEND C++ CODE.....	8
10.2	FRONTEND PYTHON CODE.....	9
10.3	SNAPSHOTS.....	12
11	REFERENCES.....	14

1. Project Title

File Founder: A Cross-Language GUI Tool for Recursive File Search in Linux Systems

2. Introduction

Keeping your digital information neat is easy thanks to the file management function in any operating system. As we keep more files and the storage space keeps growing, it becomes more important to be able to locate a particular file smoothly. Often, less technical users can't make use of find or locate, which are strong command-line utilities in Unix-based systems. Advanced users can still discover that they are inconvenient for tasks done daily. The developers have created a solution that uses both the fast C++ and the user-friendly Python graphical interface.

File Founder is a program that a user can use to search for needed files in directories of their choice. Since the backend is built in C++, it is able to quickly work through directories using system calls that get right to the point. It guarantees steady operation when dealing with large or complex hierarchy in directories. Python's Tkinter helps to make the frontend look clean and easy to use, so users can carry out their search commands easily.

Thanks to the flexible approach, the tool gets both clear instructions and effective commands. It becomes especially handy for developers, system administrators, and regular users who want to look up files in complex folders. File Founder simplifies using recursive search and improves both productivity and usability. The project is useful and shows how to write programs and control systems that work in many languages.

3. Objective

The project mainly focuses on making a desktop application that quickly and repeatedly helps users find files in the directories they specify on Linux. The project is designed to make sure that the software runs fast and also feels approachable by connecting C++ in the backend to Python in the frontend.

With this strategy, the project shows that it is possible to control a C++ compiled program through a Python user interface. Relying on the high performance of C++ in directory handling, File Founder makes sure file searches work fast and correctly within complex file systems.

One more aspect is that the project deals with common issues in command-line file search tools by making its interface clean and user-friendly. As a result, individuals with any level of knowledge can use the tool and benefit from an easy method to look for files within complicated directories.

4 . Features

The service includes valuable and sensible features to help you search for files more efficiently in a Linux operating system. The following are the important aspects of the application:

C++ backend provides a recursive search feature that goes through all the files and folders starting with the directory selected by the user. It enables easy and precise finding of files which are many layers deep.

You'll find a user-friendly and contemporary Python Tkinter GUI, spiced up with themed ttk widgets in the application. Thanks to this, users do not need to learn commands because everything is launched from the main window.

By using the subprocess module, File Founder smoothly connects Python and C++ in its code. GUI triggers the compiled C++ program, and it shows the results as they occur in real time.

The search results can be viewed in a text area that scrolls, all for easy readability. With this feature, users can check many results that are neatly presented on the same page.

The system double-checks every users' actions to keep the system running well and prevent any issues.

Detailed Reporting of Errors: The application displays proper warning messages if the C++ file is absent, the data is not proper, or other problems come up.

Made to Fit: The user interface does not allow resizing and is built to work fast and clearly, providing good performance in every system.

5. Requirements

The File Founder application can work properly only if your system meets its necessary requirements. This set of requirements allows the application to work properly and join C++ and Python parts without any incompatibilities.

System Requirements

Some examples are Ubuntu, Debian, and Fedora, which are Linux-based operating systems.

- RAM: You need at least 512 MB, but 1 GB will give better results.
- To ensure security, you should use a mobile device with a processor that is at least 1 GHz dual-core.

- The recommended Python version is 3.6 or anything higher.

This backend C++ code is compiled by using g++ (GNU C++ Compiler)

By having these necessary hardware and software, the application can perform well on systems with fewer resources.

Software Dependencies

- Python includes many important libraries.
 - tkinter is used to make the graphical user interface So your applications can have enhanced and themed GUI widgets
 - subprocess: To open and use the C++ binary created in Python
 - messagebox: To show error and confirmation messages
- C++ requirements should be met.
 - libraries used in Standard C++ including <iostream>, <dirent.h>, <sys/stat.h>, and <unistd.h>
- You can manage system permissions.
 - Users should have permission to look inside the required directories. A properly authorized user is necessary to run the application and perform a recursive search operation.

Following these criteria makes it possible for the frontend and backend parts of the application to work and communicate together without issues.

6. Installation:

To install and run File Founder, follow these step-by-step instructions to set up the required environment, compile the C++ backend, and launch the Python GUI frontend. This ensures that both the backend search engine and the user interface function correctly.

Step 1: Install Required Packages

Verify that the required libraries and development tools are installed on your Linux system.

Command:

```
sudo apt update  
  
sudo apt install g++ python3-tk
```

Explanation:

g++: The GNU C++ compiler is used to compile the backend source code **searchfile.cpp** into an executable file.

python3-tk: The GUI library Tkinter, which Python uses to create File Founder's graphical user interface, is installed by this package.

Step 2: C++ Backend

It compiling the backend component which is responsible for searching files in the system.

1. Save the provided C++ source code into a file named **searchfile.cpp**.
2. Open a terminal and direct to the directory where this file is save.

```
cd /path/to/your/project/
```

3. Compile the C++ source code using the following command:

```
g++ searchfile.cpp -o searchfile
```

Result:

An executable file named searchfile will be generated in the same directory. This binary performs the actual file search operation requested by the GUI.

Step 3: Set Up the Python Frontend

1. Save the Python GUI code into a file named **file_founder_gui.py**
2. Make sure this file is located in the same directory as the compiled **searchfile** executable.

Importance:

The Python script uses system calls to interact with the searchfile backend. Keeping both files in the same directory ensures seamless execution without path issues.

Step 4: Run the Application

Use the following command in the terminal to start the File Founder GUI:

```
python3 file_founder_gui.py
```

Result:

1 A GUI window will appear.

2 Enter the starting directory and the file name you want to search for.

3 Click the Search button to begin the operation. The GUI will call the C++ executable, display results, and handle errors if any.

Note:

If the searchfile does not run, you may need to allotment it to execute a permissions:

```
chmod +x searchfile
```

7. Usage:

File Founder is a GUI program that makes recursively searching for files on Linux convenient. Below are steps to use File Founder:

7.1. Run the Application

Open a terminal and go to the project folder, which holds the C++ compiled executable file (searchfile) and a Python GUI script file (file_founder_gui.py). Then execute the following command.

```
python3 file_founder_gui.py
```

it display some application of the main window.

7.2. Entering Search Parameters:

The GUI has two required fields:

Directory to start search: Enter the full path of a directory to start the searching (e.g. /home/user/Documents). File Founder will recursively search that directory and all its subfolders.

File name to search: Enter the exact name of the file (e.g. report.pdf, log.txt).

The tool will ensure valid syntax in each of the field before proceeding.

7.3.Search for the File

Click the "**Search File**" button. The Python frontend uses the subprocess method to invoke the C++ backend searchfile execute with the fields as arguments.

If your file is found, the full path will be in the Search Results section of the GUI.

If the file is not found, it will provide an appropriate message.

The process happens quickly as C++ is used to recursively search while the Python GUI made it very easy for the end user.

8.Troubleshooting:

Despite being designed to be user-friendly, you could still face some challenges while using File Founder. The following are examples of common problems that you might have and their possible solutions:

8.1. Executable not found error:

If the compiled C++ executable (searchfile) is not in the same directory as your Python script (file_founder_gui.py) this error comes up.

Solution:

First ensure that your C++ program compiled successfully with g++. Then search file is in the same folder as the GUI script.

8.2.Permission Deny:

The application cannot read the provided directory because of its lacks permission.

Solution:

Use `ls -l /path/to/directory` to confirm the directory's permission

Run the script with `sudo` (when necessary) or use `chmod` to update permissions.

8.3.No output or “File Not Found” when searching.

Though you have handwritten file, writing result returns 0.

Solution:

Check typos in Start Directory and File Name again.

1. The file name you entered is case-sensitive; hence, `Report.txt` is not the same as `report.txt`.

8.4. GUI does not start:

Issue: The graphical interface will not load.

Solution:

Check if `tkinter` is installed by running.

```
python3 -m tkinter
```

8.5. Segmentation Fault

Issue: The C++ backend crashes when it is attempt to access restricted directory or broken symbolic links.

Solution:

Recompile the C++ code with additional error handling.

Do not search system-protected directories unless you have no other choice.

Final Remarks:

If issues are still present please check the terminal to see if there is an error message. When dealing with Segmentation Faults, using gdb to debug the C++ code could save a fair amount of time. Just remember to always check permissions and that the paths are valid before searching.

9 Conclusion :

File Founder is an ideal combination of C++'s speed and Python's simplicity, providing a fast, easy-to-use tool for recursive file searches in Unix systems. Automating what is otherwise a manual and lengthy process can greatly increase the productivity of developers, system administrators, and general users.

This project pointed out a number of important principles of software engineering:

Performance: Often in software engineering, performance means fast code but it can mean other things as well. The C++ backend traverses directories and buffers files from disk very quickly, taking advantage of the speed C++ offers for many system operations.

Resilience: The resilience of the application includes very comprehensive error handling (access permissions, validating paths, and avoiding file system crashes).

In addition to the actual functionality of File Founder, it will serve as an excellent educational prototype for developers wanting to explore multi-language, system programming, or GUI-based applications. Some aspects of the functionality that could be added in the future include

Advanced Search Functionality: Regular expressions, partial matches, or searching for text within the contents of files.

10. Code and Snapshots

10.1 Backend C++ code

```
#include <iostream>
#include <string>
#include <dirent.h>
#include <sys/stat.h>
#include <unistd.h>

using namespace std;

void searchFile(const string& dirPath, const string& targetFile) {
    DIR* dir = opendir(dirPath.c_str());
    if (!dir) {
        perror(("Cannot open directory: " + dirPath).c_str());
        return;
    }

    struct dirent* entry;
    while ((entry = readdir(dir)) != nullptr) {
        string name = entry->d_name;

        // Skip "." and ".."
        if (name == "." || name == "..")
            continue;

        string fullPath = dirPath + "/" + name;
```

```

    struct stat fileStat;
    if (stat(fullPath.c_str(), &fileStat) == -1) {
        perror(("Stat failed for: " + fullPath).c_str());
        continue;
    }

    if (S_ISDIR(fileStat.st_mode)) {
        // Recurse into subdirectory
        searchFile(fullPath, targetFile);
    } else if (name == targetFile) {
        cout << "Found: " << fullPath << endl;
    }
}

closedir(dir);
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cout << "Usage: " << argv[0] << " <start_directory> <file_name>" <<
endl;
        return 1;
    }

    string startDir = argv[1];
    string targetFile = argv[2];

    searchFile(startDir, targetFile);

    return 0;
}

```

10.2 Frontend python Code

```

import subprocess
import tkinter as tk
from tkinter import messagebox, ttk # Import ttk for themed widgets

def search_file():
    directory = dir_entry.get().strip()
    filename = file_entry.get().strip()

```

```

if not directory or not filename:
    messagebox.showerror("Error", "Both fields are required.")
    return

try:
    result = subprocess.run(
        ["/searchfile", directory, filename],
        capture_output=True,
        text=True
    )

    if result.returncode == 0:
        output = result.stdout.strip()
        if output:
            output_text.config(state='normal')
            output_text.delete("1.0", tk.END)
            output_text.insert(tk.END, output)
            output_text.config(state='disabled')
            messagebox.showinfo("Search Result", "File found!")
        else:
            messagebox.showinfo("Search Result", "File not found.")
    else:
        messagebox.showerror("Error", result.stderr.strip())
except FileNotFoundError:
    messagebox.showerror("Error", "C++ executable './searchfile' not found.
Make sure it's compiled and in the same directory.")
except Exception as e:
    messagebox.showerror("Exception", str(e))

# --- GUI setup ---
root = tk.Tk()
root.title("Advanced File Search")
root.geometry("600x500")
root.resizable(False, False)

# Apply a modern theme
style = ttk.Style()
style.theme_use("clam")

main_frame = ttk.Frame(root, padding="20 20 20 20")
main_frame.pack(fill=tk.BOTH, expand=True)

# Input section
input_frame = ttk.LabelFrame(main_frame, text="Search Parameters", padding="15
15 15 15")

```

```

input_frame.pack(pady=10, fill=tk.X)

tk.Label(input_frame, text="Start Directory (eg: /home): ", font=("Arial", 10,
"bold")).pack(anchor=tk.W, pady=(0, 2))
dir_entry = ttk.Entry(input_frame, width=70)
dir_entry.pack(fill=tk.X, expand=True)

tk.Label(input_frame, text="File Name to Search (eg: file.txt):",
font=("Arial", 10, "bold")).pack(anchor=tk.W, pady=(10, 2))
file_entry = ttk.Entry(input_frame, width=70)
file_entry.pack(fill=tk.X, expand=True)

# Search button
search_button = ttk.Button(main_frame, text="Search File", command=search_file,
style="Accent.TButton")
search_button.pack(pady=15)

# Customize button style
style.map("Accent.TButton",
    foreground=[('pressed', 'white'), ('active', 'white')],
    background=[('pressed', '!focus', '#0056b3'), ('active', '#007bff')],
    relief=[('pressed', 'groove'), ('!pressed', 'ridge')]
)
style.configure("Accent.TButton",
    font=("Arial", 11, "bold"),
    background="#007bff",
    foreground="white",
    padding=10,
    relief="raised"
)

# Output section
output_frame = ttk.LabelFrame(main_frame, text="Search Results", padding="15 15
15 15")
output_frame.pack(pady=10, fill=tk.BOTH, expand=True)

output_text = tk.Text(output_frame, height=10, wrap="word", state='disabled',
    font=("Consolas", 10), bg="#f0f0f0", fg="#333",
    relief="sunken", borderwidth=2)
output_text.pack(fill=tk.BOTH, expand=True)

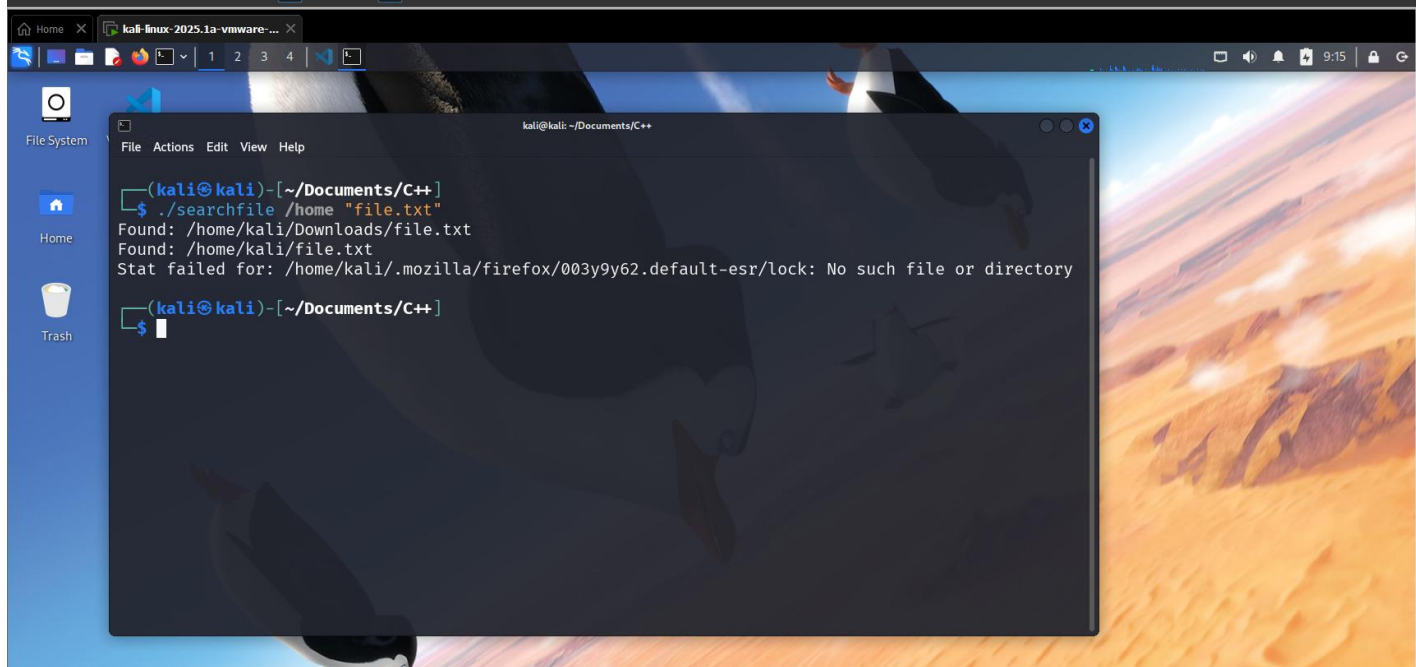
# Add a scrollbar to the output text area
scrollbar = ttk.Scrollbar(output_frame, command=output_text.yview)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
output_text.config(yscrollcommand=scrollbar.set)

```

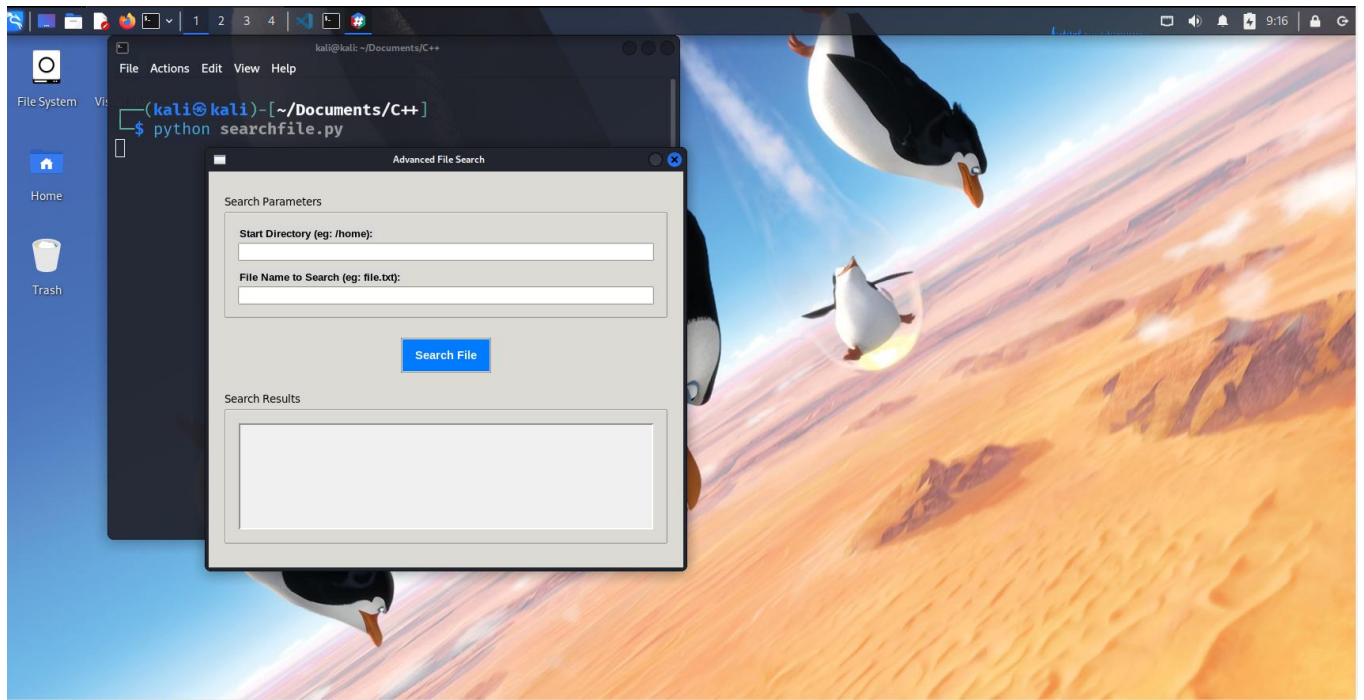
```
root.mainloop()
```

10.3 Snapshots

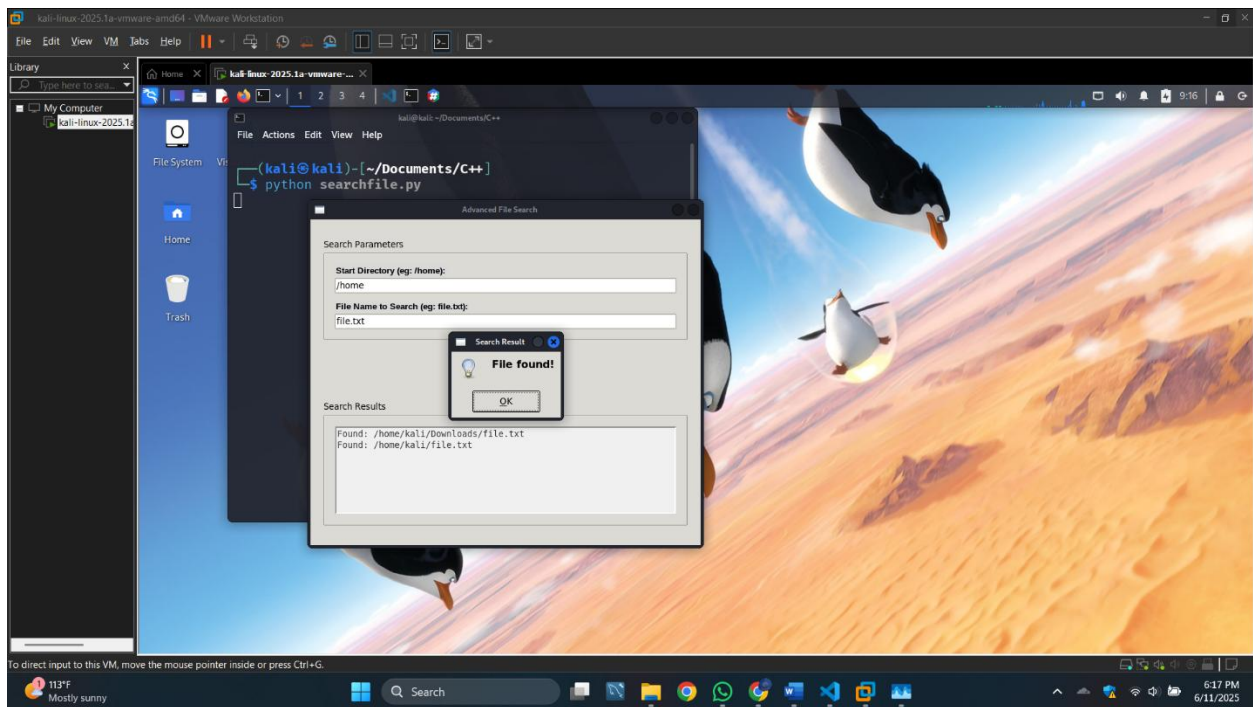
i. Terminal working of C++ code



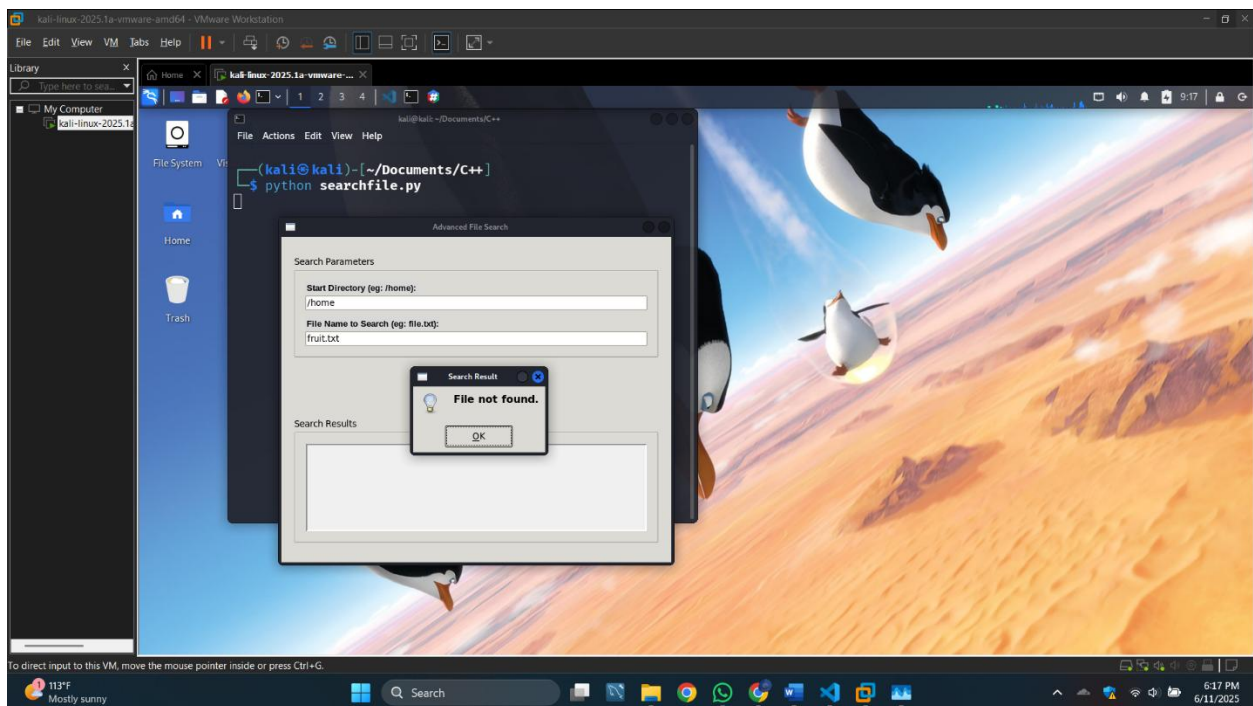
ii. GUI main window



iii. Successful file search



iv. Unsuccessful file search



11. References

- C++ `<dirent.h>` documentation
- [Tkinter GUI Documentation](#)
- [Stack Overflow – Python-C++ integration with subprocess]
- Linux manual pages (`man dir`, `man g++`, `man python3`)